

UNINTER - CENTRO UNIVERSITÁRIO INTERNACIONAL

Magno Rodrigues Ramos Filho - RU 5023493

**PROJETO MULTIDISCIPLINAR - TRILHA
BACK-END**

**API DE PEDIDOS DA REDE RAÍZES DO
NORDESTE**

GOIÂNIA - GOIÁS
2026

Sumário

1	Introdução e objetivos	1
2	Análise do problema e requisitos	1
3	Modelagem e arquitetura	1
3.1	Casos de uso	2
3.2	Diagrama de classes	3
3.3	Modelo de dados	4
3.4	Sequência do fluxo crítico	5
3.5	Especificação do caso de uso crítico	5
4	API e regras de negócio	6
5	Segurança, privacidade e LGPD	6
6	Entrega técnica e execução	7
7	Plano de testes	7
8	Conclusão	8
	Referências	8
	Apêndice A — Requisitos e prioridades	9
	Apêndice B — Contratos dos endpoints	13
	Apêndice C — Cenários e resultados esperados	20
	Apêndice D — Evidências de execução	29
	Apêndice E — Decisões técnicas	32

1 Introdução e objetivos

Este projeto propõe e implementa um back-end para uma rede de lanchonetes que atende clientes por canais digitais e presenciais e opera em múltiplas unidades. O objetivo é demonstrar um fluxo persistente e reproduzível de pedido, reserva de estoque, pagamento simulado e atualização de status, com contratos de API, perfis de acesso e documentação técnica.

O recorte prioriza o fluxo crítico e a rastreabilidade. A API foi desenvolvida em Python com FastAPI, SQLAlchemy e SQLite; as decisões de dependência e organização estão detalhadas no README do projeto.

2 Análise do problema e requisitos

O sistema precisa identificar a origem de cada pedido e controlar saldo por unidade. O estoque é reservado durante a espera pelo pagamento. Uma recusa cancela o pedido e devolve as quantidades. A aprovação encaminha o pedido para preparação; COZINHA pode marcá-lo como pronto e ATENDENTE pode concluir a entrega. O cliente consulta apenas os próprios pedidos, enquanto usuários internos têm acesso conforme o perfil.

O requisito multicanal é representado por `canalPedido`, obrigatório na criação e disponível como filtro. O preço de cada item é preservado como fotografia no momento da compra. As prioridades e o estado de cada requisito estão na [análise de requisitos](#).

O MVP implementa autenticação, autorização por perfil, catálogo, estoque, pedidos, pagamento mock, fidelidade opcional e auditoria. Promoções permanecem como proposta; não são anunciadas como função disponível.

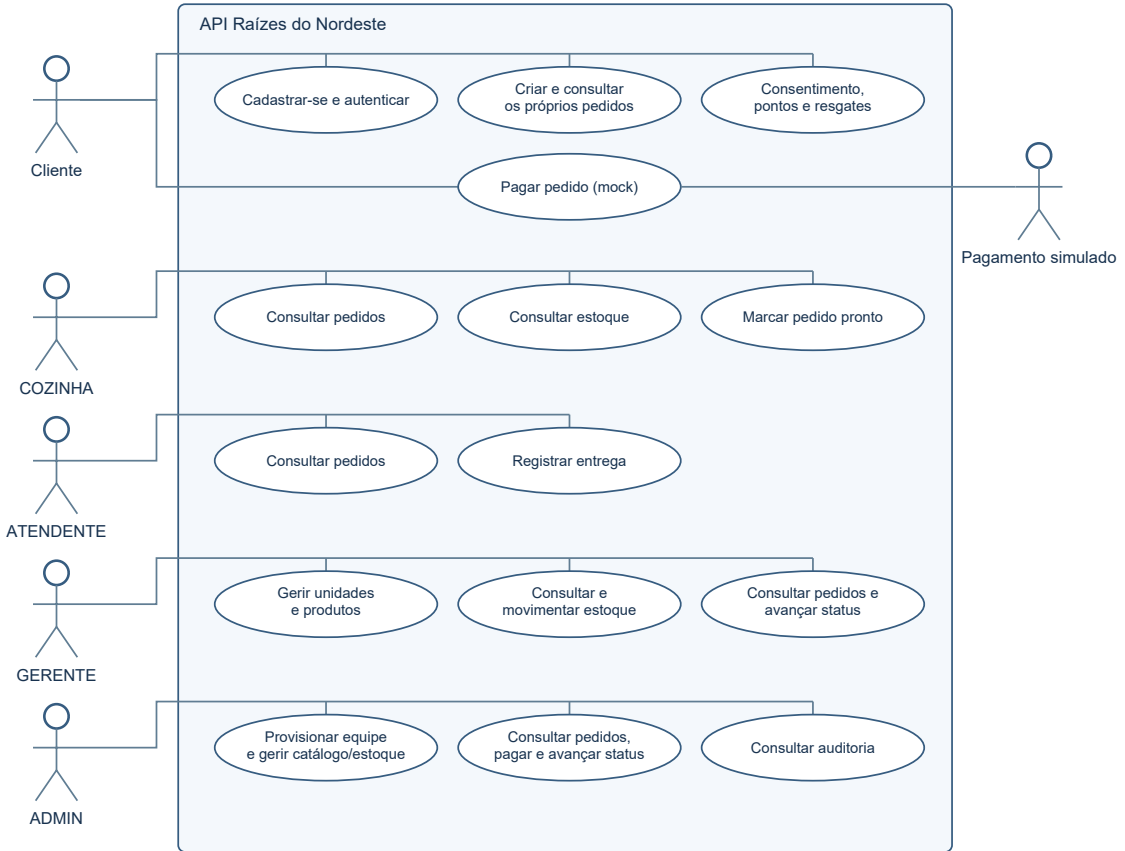
3 Modelagem e arquitetura

O código separa API, aplicação, domínio e infraestrutura. Rotas validam e encaminham solicitações; casos de uso aplicam as regras; tipos do domínio descrevem perfis, canais e estados; SQLAlchemy e Alembic persistem e versionam os dados.

3.1 Casos de uso

Casos de uso — Raízes do Nordeste

Perfis de acesso e participante de pagamento simulado



O ator de pagamento representa a integração conceitual. Nesta versão, o resultado é produzido por um mock interno. Cada perfil interno também se autentica. As associações estão agrupadas por ator para facilitar a leitura.

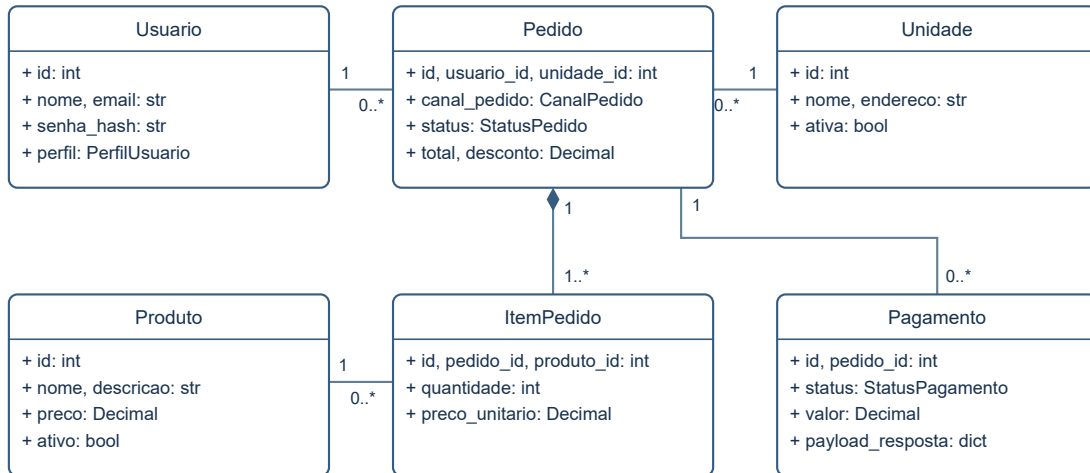
O diagrama distingue os perfis COZINHA e ATENDENTE, além de GERENTE e ADMIN. O pagamento é mostrado como participante externo esperado na integração, mas o MVP executa somente um simulador interno, sem conexão de rede.

3.2 Diagrama de classes

Classes e associações — Raízes do Nordeste

Modelos implementados • atributos públicos principais • multiplicidades UML

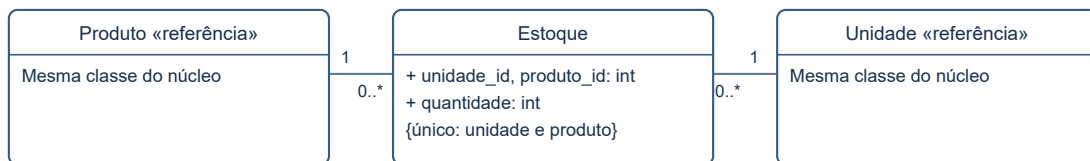
01 Pedidos e pagamento



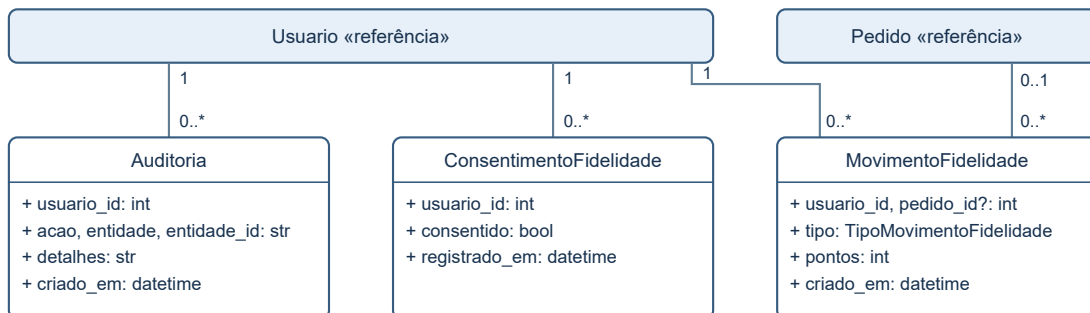
Composição: o item pertence ao pedido. O preço unitário é preservado no momento da compra.

Pagamento: o banco admite vários registros; o fluxo sequencial bloqueia nova tentativa após processar.

02 Estoque por unidade



03 Auditoria e fidelidade



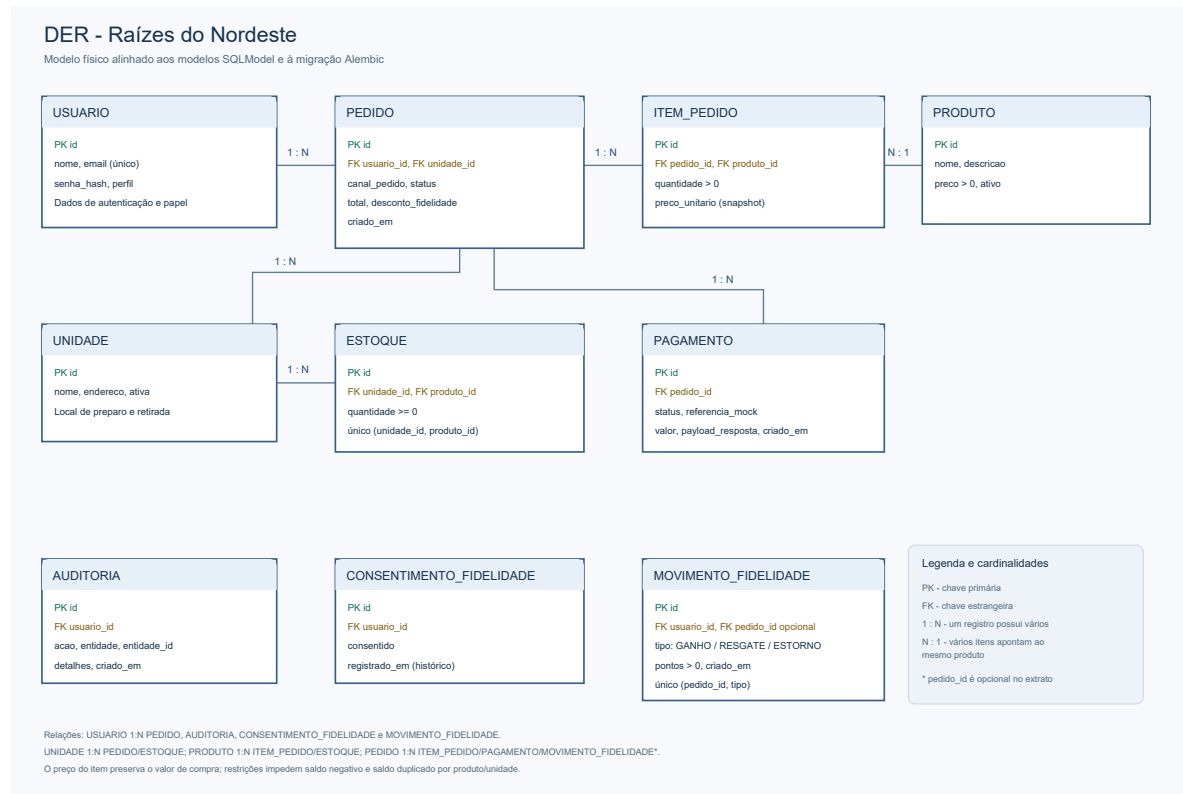
As referências repetem classes para evitar cruzamentos. Cada classe persistida possui id: int.

Pedido.desconto representa desconto_fidelidade. Movimentos são únicos por pedido e tipo.

As regras são funções da camada de aplicação; os modelos SQLAlchemy não declaram métodos de negócio.

As classes refletem os modelos SQLAlchemy persistidos e suas associações. Chaves primárias, chaves estrangeiras, tipos relevantes e restrições de estoque e preço estão destacados.

3.3 Modelo de dados

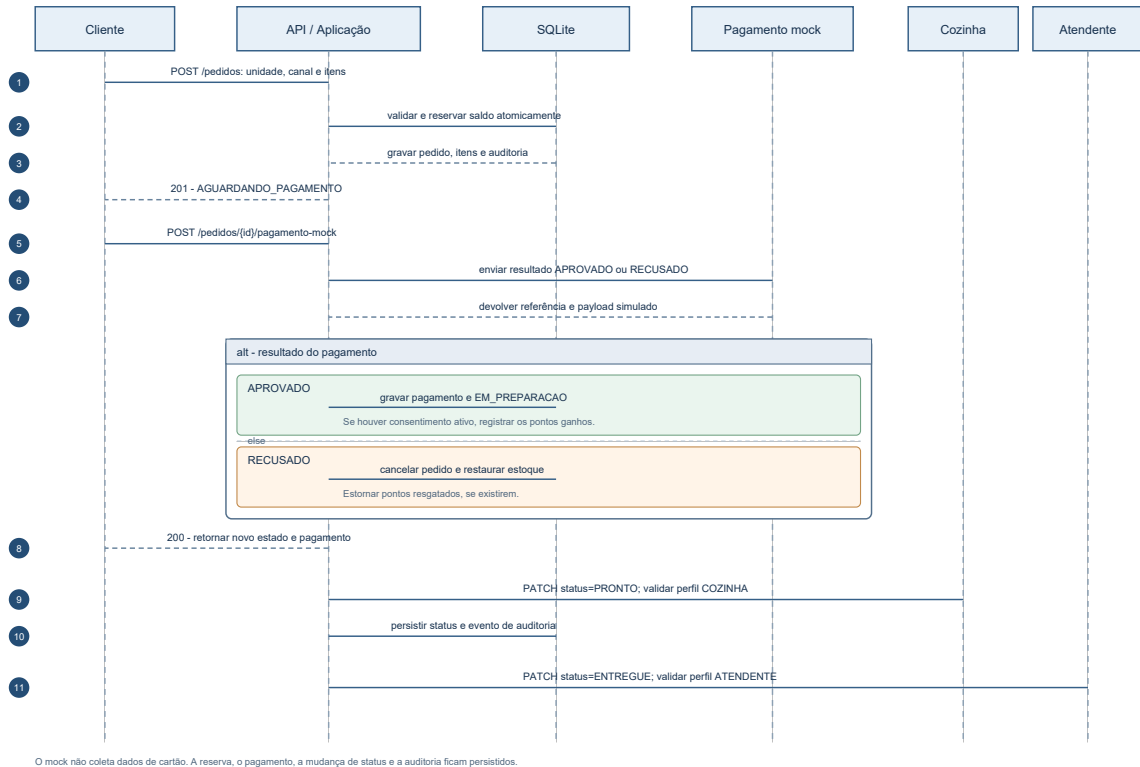


As entidades cobrem usuários, unidades, produtos, saldos, pedidos, itens, pagamentos, auditoria, consentimentos e movimentos de fidelidade. Chaves estrangeiras e restrições de integridade conectam a modelagem ao banco implementado.

3.4 Sequência do fluxo crítico

Sequência do fluxo crítico

Pedido multicanal - reserva de estoque - pagamento mock - preparo e entrega



3.5 Especificação do caso de uso crítico

UC01 — Criar pedido e processar pagamento. Ator principal: cliente autenticado. Participam a API, a aplicação, o banco SQLite e o simulador de pagamento. O caso começa quando o cliente envia unidade, canal, produtos e quantidades.

Pré-condições. O token é válido; a unidade está ativa; cada produto existe, está ativo e tem saldo suficiente; há pelo menos um item com quantidade positiva; o canal pertence a APP, TOTEM, BALCAO, PICKUP OU WEB.

Fluxo principal. A API valida identidade e contrato. A aplicação confere unidade, produto e saldo, calcula o valor e registra o pedido, os itens com preços unitários congelados, a reserva de estoque e a auditoria em uma transação. O cliente recebe o pedido em AGUARDANDO_PAGAMENTO. Quando o mock aprova, o pagamento e sua referência fictícia são registrados, o pedido passa a EM_PREPARACAO e, com consentimento ativo, os pontos do valor pago são lançados. COZINHA marca o pedido PRONTO; ATENDENTE o marca ENTREGUE.

Pós-condição de aprovação. O pedido está em preparação, o pagamento está aprovado, o estoque reservado continua abatido e os pontos aplicáveis estão registrados.

Alternativa de recusa. O pagamento é registrado como recusado; o pedido é cancelado, o estoque é devolvido e eventual resgate de pontos é estornado.

Exceções. Unidade ou produto ausente resulta em 404; saldo insuficiente, pagamento repetido ou transição de estado incompatível resulta em 409; contrato inválido resulta em 422; perfil sem permissão resulta em 403. Em falha de gravação, a transação é revertida e a resposta inclui identificador de requisição sem expor detalhes internos.

Regras de negócio. Canal é obrigatório; o cliente só acessa seus pedidos; a reserva por unidade/produto não pode produzir saldo negativo; falha em qualquer item desfaz toda a criação; o preço registrado não muda com o catálogo; cada pedido admite somente um processamento de pagamento; recusa devolve a reserva; COZINHA conclui preparo e ATENDENTE conclui entrega; fidelidade depende de consentimento; o pagamento simulado não coleta dados de cartão; operações sensíveis geram auditoria sem senha ou token.

Os critérios completos e numerados estão em [fluxo-critico.md](#).

4 API e regras de negócio

A API documenta autenticação, catálogo, estoque, pedidos, pagamento, fidelidade e auditoria na [referência de endpoints](#). O Swagger/OpenAPI local é servido em /docs, e o JSON OpenAPI em /openapi.json.

As entradas são validadas por contrato. canalPedido aceita APP, TOTEM, BALCAO, PICKUP OU WEB. O pedido exige itens válidos e saldo suficiente; quantidades e preços passam por validações. O pagamento mock registra uma referência e um payload fictícios, sem receber dados de cartão. As falhas usam envelope JSON uniforme com identificador de requisição.

5 Segurança, privacidade e LGPD

O cadastro público cria apenas perfil CLIENTE. Perfis internos são provisionados por ADMIN; os endpoints verificam o token e o perfil adequado. Senhas são armazenadas com hash Argon2 e tokens JWT têm prazo de validade. Respostas públicas de usuário não contêm hash nem senha.

São coletados nome e e-mail para cadastro, autenticação e atendimento dos pedidos. A hipótese de execução de contrato pode ser aplicável às operações necessárias à prestação solicitada; essa decisão cabe ao controlador, considerando o caso concreto. A adesão à fidelidade é opcional: o aceite ou a revogação é registrado com data e não impede o uso das funções de pedido.

O sistema evita coletar cartão de pagamento e orienta que logs não incluam senhas, tokens ou dados pessoais desnecessários. A retenção deve seguir finalidade, obrigações legais e política definida pelo controlador. Ao término da finalidade, deve-se avaliar eliminação ou anonimização e atender os direitos aplicáveis do titular. O documento de [decisões de projeto](#) registra esses limites e as referências legais consultadas.

6 Entrega técnica e execução

Repositório público do projeto:
<https://github.com/MagnoRamos06/api-raizes-do-nordeste>.

O repositório contém código, migração inicial, `.env.example`, README, coleção Postman, plano de cenários e diagramas. O README explica instalação, configuração, criação do banco, seed, execução e acesso ao Swagger. A migração cria o esquema de forma versionada; a API não cria tabelas automaticamente na inicialização.

O relatório acadêmico em PDF identifica instituição, estudante, RU, cidade e ano. A migração, o seed e a coleção Postman foram executados localmente; o resumo verificável está na [evidência de testes](#). O código, a documentação técnica, os diagramas, a migração e a coleção Postman estão publicados no repositório público do projeto.

7 Plano de testes

O plano descreve cenários positivos e negativos de autenticação, autorização, validação, estoque, pedido, pagamento recusado, fidelidade e auditoria. A coleção Postman organiza as chamadas do fluxo e scripts para guardar tokens e IDs.

Em 25/09/2026, a coleção foi executada contra uma instância local da API com um banco SQLite limpo: 35 chamadas e 35 scripts de teste passaram sem falhas. A evidência registra os fluxos positivos e negativos sem incluir credenciais.

8 Conclusão

O MVP fecha o fluxo pedido → pagamento mock → preparação → entrega, com persistência relacional, estoque por unidade, multicanalidade e autorização por perfil. Validações e erros padronizados tornam o comportamento mais previsível; auditoria registra operações relevantes. A fidelidade foi incluída como participação voluntária, com histórico de consentimento e estorno de pontos quando necessário.

Campanhas promocionais e integração com um provedor real de pagamento permanecem fora do escopo implementado. Essa priorização mantém o foco em demonstrar o fluxo principal com consistência. DER, casos de uso, sequência, requisitos e endpoints descrevem o mesmo escopo que o código; a coleção Postman reproduz o percurso principal e seus cenários negativos, executados com sucesso no ambiente local.

O PDF inclui apêndices com requisitos priorizados, contratos dos endpoints, cenários de teste, resultados esperados e evidências de execução. As tabelas são apresentadas como fichas para preservar a leitura.

Referências

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709compilado.htm. Acesso em: 24 set. 2026.

AUTORIDADE NACIONAL DE PROTEÇÃO DE DADOS. Direitos dos titulares. Brasília, DF: ANPD. Disponível em: <https://www.gov.br/anpd/pt-br/assuntos/titular-de-dados>. Acesso em: 24 set. 2026.

Apêndice A — Requisitos e prioridades

Contexto e objetivo

A rede Raízes do Nordeste atende por canais digitais e presenciais e opera mais de uma unidade. O back-end deve registrar a origem do pedido, consultar disponibilidade por unidade e coordenar pagamento e preparo sem permitir que um pedido ultrapasse o saldo disponível.

Atores

Ator	Responsabilidade no sistema
Cliente	Cadastrar-se, autenticar-se, criar e consultar os próprios pedidos, pagar pelo mock e escolher se participa do programa de fidelidade.
COZINHA	Consultar pedidos e estoque e marcar como pronto os pedidos em preparação.
ATENDENTE	Consultar pedidos e marcar como entregue os pedidos prontos.
GERENTE	Gerir catálogo e estoque e avançar as duas transições de status.
ADMIN	Provisionar contas internas, gerir catálogo/estoque, consultar auditoria e administrar status.

Requisitos funcionais priorizados

ID	RF01
Prioridade	P0
Requisito	Cadastrar clientes e autenticar com token; nunca expor a senha armazenada.
Estado no MVP	Implementado

ID	RF02
Prioridade	P0
Requisito	Restringir operações por perfil e permitir provisionamento interno apenas por ADMIN.
Estado no MVP	Implementado

ID	RF03
Prioridade	P0
Requisito	Manter unidades, produtos e saldo de estoque por unidade.
Estado no MVP	Implementado

ID	RF04
Prioridade	P0
Requisito	Criar pedidos com canal Pedido, validar unidade/produto/quantidade/saldo e guardar preço no momento da compra.
Estado no MVP	Implementado

ID	RF05
Prioridade	P0
Requisito	Reservar estoque de forma condicional; devolver o saldo em pagamento recusado.
Estado no MVP	Implementado

ID	RF06
Prioridade	P0
Requisito	Simular aprovação ou recusa do pagamento e registrar status, valor, referência e resposta mock.
Estado no MVP	Implementado

ID	RF07
Prioridade	P0
Requisito	Aplicar transições de pedido autorizadas conforme o perfil.
Estado no MVP	Implementado

ID	RF08
Prioridade	P0
Requisito	Consultar pedidos com filtros por canal/status e paginação; clientes veem somente os próprios.
Estado no MVP	Implementado

ID	RF09
Prioridade	P1
Requisito	Oferecer fidelidade opcional com consentimento histórico, acúmulo, resgate e estorno.
Estado no MVP	Implementado

ID	RF10
Prioridade	P1
Requisito	Auditar ações sensíveis e permitir consulta restrita a ADMIN.
Estado no MVP	Implementado

ID	RF11
Prioridade	P2
Requisito	Gerir campanhas promocionais por período, produtos/unidades e regra de desconto.
Estado no MVP	Proposta; não implementado

Requisitos não funcionais

ID	Requisito	Decisão / evidência
RNF01	Persistência relacional real e alterações reproduzíveis.	SQLite com SQLAlchemy e migração Alembic versionada.
RNF02	Integridade de valores e quantidades.	Decimal monetário, validações de contrato e restrições SQL para preço, quantidade e saldo.
RNF03	Segurança das credenciais e endpoints.	Hash Argon2, JWT, validação de perfil e .env local fora do repositório.
RNF04	Respostas previsíveis em falhas.	Envelope comum com código, mensagem, detalhes, rota, horário e requestId.
RNF05	Evolução e manutenção do código.	Separação API, aplicação, domínio e infraestrutura; contratos e documentação dedicados.
RNF06	Privacidade proporcional ao escopo.	Minimização de dados, fidelidade opcional, auditoria sem credenciais e estratégia de retenção documentada.
RNF07	Correção reproduzível.	README, Swagger/OpenAPI, DER, plano e coleção de cenários Postman.

Desempenho, disponibilidade e falhas de integração

RNF08 — Desempenho em horários de pico: listagens principais usam paginação e os campos de consulta têm índices. Testes de carga e dimensionamento de capacidade permanecem como evolução; a execução local não comprova desempenho em produção.

RNF09 — Disponibilidade: a rota /health permite verificar a aplicação. Supervisão de processo, cópias de segurança e restauração do banco são medidas propostas para uma futura implantação. O MVP não estabelece acordo de nível de serviço.

RNF10 — Falhas de pagamento: o mock permite aprovação e recusa, preservando transação, estoque e fidelidade. Uma integração real exigirá tempo limite, tratamento de indisponibilidade, reconciliação e identificador de idempotência. Não há chamadas a um provedor externo nesta versão.

Recorte do MVP e justificativas

O caminho P0 é pedido → reserva de estoque → pagamento mock → preparação → entrega. Ele percorre autenticação, regras de negócio, persistência, multicanalidade e autorização de ponta a ponta. Fidelidade foi incluída como requisito P1 por sua relevância ao cenário, com participação voluntária para reduzir coleta desnecessária. A integração de pagamento é deliberadamente um mock: permite demonstrar sucesso e recusa sem armazenar dados de cartão nem depender de serviço externo.

Campanhas promocionais permanecem como proposta P2; a API não deve anunciar essa funcionalidade como disponível. O recorte deixa a solução demonstrável sem acrescentar lógica comercial ainda não necessária ao fluxo mínimo do roteiro.

Apêndice B — Contratos dos endpoints

Base local: `http://127.0.0.1:8000`. A documentação interativa gerada pelo FastAPI fica em `/docs`; OpenAPI em `/openapi.json`. Para rotas protegidas, envie `Authorization: Bearer <token>`.

Os corpos e respostas usam nomes JSON em camelCase quando indicado no contrato. Campos monetários são decimais em reais. Todas as falhas tratadas pela API seguem este envelope:

```
{
  "error": "VALIDATION_ERROR",
  "message": "Um ou mais campos são inválidos.",
  "details": [{"field": "canalPedido", "issue": "Campo obrigatório."}],
  "timestamp": "2026-09-24T12:00:00+00:00",
  "path": "/pedidos",
  "requestId": "identificador-da-requisicao"
}
```

Autenticação

Método e rota	POST /auth/cadastro
Acesso	Público
Entrada principal	<code>{"nome": "Ana Silva", "email": "ana@example.com", "senha": "senha-segura", "consentimento_fidelidade": true}</code>
Sucesso	201: usuário público com id, nome, email, perfil=CLIENTE
Erros relevantes	409 e-mail cadastrado; 422 campos inválidos

Método e rota	POST /auth/login
Acesso	Público
Entrada principal	<code>{"email": "ana@example.com", "senha": "senha-segura"}</code>
Sucesso	200: <code>{"access_token": "...", "token_type": "bearer"}</code>
Erros relevantes	401 credenciais inválidas

Método e rota	GET /auth/me
Acesso	Bearer, qualquer perfil
Entrada principal	—
Sucesso	200: dados públicos do usuário autenticado
Erros relevantes	401 token ausente, inválido ou expirado

Método e rota	POST /auth/usuarios-operacionais
Acesso	ADMIN
Entrada principal	Cadastro com perfil igual a ATENDENTE, COZINHA ou GERENTE
Sucesso	201: usuário público
Erros relevantes	401, 403, 409 e-mail duplicado, 422 perfil/campos inválidos

O cadastro público sempre cria `CLIENTE`; só `ADMIN` pode provisionar os perfis operacionais.

Exemplos de resposta de sucesso (IDs e token ilustrativos):

```
{
  "id": 1,
  "nome": "Ana Silva",
  "email": "ana@example.com",
  "perfil": "CLIENTE"
}
```

```
{
  "access_token": "<jwt>",
  "token_type": "bearer"
}
```

O primeiro objeto é usado no cadastro, na consulta `/auth/me` e no provisionamento operacional (com o perfil correspondente).

Catálogo e estoque

Método e rota	GET /unidades
Acesso	Público
Entrada principal	—
Sucesso	200: lista de unidades ativas
Erros relevantes	—

Método e rota	POST /unidades
Acesso	GERENTE ou ADMIN
Entrada principal	{ "nome": "Centro", "endereco": "Rua Exemplo, 100" }
Sucesso	201: unidade criada
Erros relevantes	401, 403, 422

Método e rota	GET /produtos?page=1&limit=20
Acesso	Público
Entrada principal	page ≥ 1; limit entre 1 e 100
Sucesso	200: lista paginada de produtos ativos
Erros relevantes	422 paginação inválida

Método e rota	POST /produtos
Acesso	GERENTE ou ADMIN
Entrada principal	{"nome":"Sanduíche","descricao":"Pão, queijo e tomate","preco":"25.00"}
Sucesso	201: produto criado
Erros relevantes	401, 403, 422

Método e rota	GET /unidades/{unidade_id}/cardapio
Acesso	Público
Entrada principal	ID da unidade no path
Sucesso	200: produtos ativos com saldo positivo, preço e disponibilidade
Erros relevantes	404 unidade ausente/inativa

Método e rota	POST /unidades/{unidade_id}/estoque/{produto_id}/movimentacoes
Acesso	GERENTE ou ADMIN
Entrada principal	{"tipo":"ENTRADA","quantidade":20} ou SAIDA
Sucesso	200: {"unidadeId":1,"produtoId":2,"quantidade":20}
Erros relevantes	401, 403, 404, 409 saída maior que saldo, 422

Método e rota	GET /unidades/{unidade_id}/estoque
Acesso	COZINHA, GERENTE ou ADMIN
Entrada principal	ID da unidade no path
Sucesso	200: saldos da unidade
Erros relevantes	401, 403, 404 unidade inexistente

Exemplos de respostas: GET /unidades retorna [{"id":1,"nome":"Centro","endereço":"Rua Exemplo, 100","ativa":true}]; cadastrar

unidade retorna o mesmo objeto. GET /produtos retorna [{"id":2,"nome":"Sanduíche","descricao":"Pão, queijo e tomate","preco":"25.00","ativo":true}]; cadastrar produto retorna o mesmo objeto. O cardápio retorna [{"produtoId":2,"nome":"Sanduíche","descricao":"Pão, queijo e tomate","preco":"25.00","disponivel":true}]. Consulta de estoque retorna [{"unidadeId":1,"produtoId":2,"quantidade":20}]; movimentação retorna um objeto com esse formato.

Pedidos e pagamento simulado

Método e rota	POST /pedidos
Acesso	CLIENTE
Entrada principal	{"unidadeId":1,"canalPedido":"TOTEEM","itens":[{"produtoId":2,"quantidade":1}]}
Sucesso	201: pedido em AGUARDANDO_PAGAMENTO, itens com preço congelado e total
Erros relevantes	401, 404 unidade/produto, 409 estoque insuficiente, 422 contrato inválido

Método e rota	POST /pedidos/{pedido_id}/pagamento-mock
Acesso	CLIENTE dono do pedido ou ADMIN
Entrada principal	{"resultado":"APROVADO"} ou {"resultado":"RECUSADO"}
Sucesso	200: pedido e pagamento simulado; aprovado inicia preparação, recusado cancela e devolve estoque
Erros relevantes	401, 403, 404, 409 pedido fora do estado esperado, 422

Método e rota	PATCH /pedidos/{pedido_id}/status
Acesso	COZINHA/GERENTE/ADMIN para PRONTO; ATENDENTE/GERENTE/ADMIN para ENTREGUE
Entrada principal	{"status":"PRONTO"} ou {"status":"ENTREGUE"}
Sucesso	200: pedido com status atualizado
Erros relevantes	401, 403, 404, 409 transição incompatível, 422

Método e rota	GET /pedidos?canalPedido=TOTEM&status=AGUARDANDO_PAGAMENTO&page=1&limit=10
Acesso	Bearer; CLIENTE vê apenas os próprios, equipe vê todos
Entrada principal	Filtros opcionais canalPedido, status, page ≥ 1, limit entre 1 e 100
Sucesso	200: lista de pedidos
Erros relevantes	401, 422 filtro/paginação inválidos

Estados previstos: AGUARDANDO_PAGAMENTO → EM_PREPARACAO → PRONTO → ENTREGUE; uma recusa de pagamento leva a CANCELADO. O pagamento é somente um mock e não processa dados de cartão.

Exemplo de resposta de pedido criado ou atualizado:

```
{
  "id": 3,
  "unidadeId": 1,
  "canalPedido": "TOTEM",
  "status": "AGUARDANDO_PAGAMENTO",
  "total": "25.00",
  "descontoFidelidade": "0.00",
  "itens": [{"produtoId": 2, "quantidade": 1, "precoUnitario": "25.00"}],
  "pagamento": null
}
```

No pagamento, pagamento passa a incluir status, referencia, valor e payload; a lista de pedidos devolve uma lista dos objetos de pedido. Os valores permitidos para canalPedido são APP, TOTEM, BALCAO, PICKUP e WEB.

Fidelidade e consentimento

Método e rota	GET /fidelidade/me
Acesso	CLIENTE
Entrada principal	—
Sucesso	200: consentimento atual, saldo e até 50 movimentos recentes
Erros relevantes	401, 403 perfil diferente de cliente

Método e rota	POST /fidelidade/consentimento
Acesso	CLIENTE
Entrada principal	{"consentido":true} ou false
Sucesso	200: resumo atualizado; cada escolha é registrada
Erros relevantes	401, 403

Método e rota	POST /fidelidade/resgates
Acesso	CLIENTE
Entrada principal	{"pedidoId":3, "pontos":100}; múltiplos de 100
Sucesso	200: pedido, pontos usados, desconto e total após desconto
Erros relevantes	401, 403, 404, 409 consentimento/saldo/estado/resgate inválido, 422

Com consentimento ativo, cada R\$ 1,00 pago gera 1 ponto; 100 pontos geram R\$ 5,00 de desconto. Resgate ocorre antes do pagamento e é estornado quando o mock recusa o pagamento.

Exemplos de sucesso: o resumo retorna {"consentido":true, "pontos":100, "extrato":[{"tipo":"GANHO", "pontos":100, "pedidoId":2, "criadoEm":"2026-09-24T12:00:00+00:00"}]}; o resgate retorna {"pedidoId":3, "pontosResgatados":100, "desconto":"5.00", "totalAtualizado":"20.00"}. Atualizar consentimento retorna o resumo no mesmo formato.

Auditoria e saúde

Método e rota	GET /auditoria?entidade=PEDIDO&entidade_id=3&page=1&limit=50
Acesso	ADMIN
Entrada principal	Filtros opcionais; page ≥ 1; limit entre 1 e 200
Sucesso	200: eventos mais recentes primeiro
Erros relevantes	401, 403, 422 paginação inválida

Método e rota	GET /health
Acesso	Público
Entrada principal	—
Sucesso	200: {"status":"ok"}
Erros relevantes	—

Auditoria retorna uma lista como [{"id":8,"usuarioId":1,"acao":"PEDIDO_CRIADO","entidade":"PEDIDO","entidadeId":"3","detalhes":"canal=TOTEM;total=25.00","criadoEm":"2026-09-24T12:00:00+00:00"}].

Códigos usados

200 consulta/atualização bem-sucedida; 201 recurso criado; 401 sem autenticação válida; 403 perfil sem permissão; 404 recurso ausente; 409 conflito com regra de negócio ou unicidade; 422 entrada inválida; 500 falha inesperada com mensagem genérica e `requestId` para rastreo.

Validação e documentação de erros

Nomes e endereços são normalizados antes da validação de tamanho. Produto e unidade exigem nome com pelo menos dois caracteres; o endereço exige cinco caracteres após a remoção de espaços externos. As mensagens de validação são apresentadas em português. O OpenAPI documenta o modelo `RespostaErro`, seus detalhes e os códigos de erro aplicáveis a cada operação.

Apêndice C — Cenários e resultados esperados

Preparação e execução

1. Configure `.env`, aplique `alembic upgrade head` e execute `python -m app.seed`.
2. Inicie a API e confirme que `http://127.0.0.1:8000/docs` está acessível.
3. Importe `postman/raizes-api.postman_collection.json` e `postman/ambiente-local.template.json` no Postman.
4. Preencha `adminEmail` e `adminPassword` com os valores usados no seed. Selecione esse ambiente.
5. Execute a coleção na ordem apresentada; os scripts guardam tokens e IDs de resposta.

O primeiro usuário cliente consente com fidelidade, compra de demonstração de R\$ 100,00, recebe 100 pontos, aplica R\$ 5,00 no pedido seguinte e recebe os pontos de volta após a recusa simulada.

Cenários executáveis

ID	T01
Cenário / endpoint	Cadastro de cliente — POST <code>/auth/cadastro</code>
Pré-condição e entrada	Seed aplicado; consentimento opcional aceito
Resultado esperado	201, perfil CLIENTE, senha ausente da resposta
Evidência na coleção	T01 - Cadastrar cliente com consentimento opcional

ID	T02
Cenário / endpoint	Login cliente — POST <code>/auth/login</code>
Pré-condição e entrada	Conta T01
Resultado esperado	200 e <code>access_token</code>
Evidência na coleção	T02 - Login do cliente

ID	T03
Cenário / endpoint	Consultar unidade — GET /unidades
Pré-condição e entrada	Seed aplicado
Resultado esperado	200 e ID disponível
Evidência na coleção	T03 - Listar unidades e guardar ID do seed

ID	T04
Cenário / endpoint	Cardápio da unidade — GET /unidades/{id}/cardapio
Pré-condição e entrada	Unidade ativa com produto em estoque
Resultado esperado	200 e lista de itens disponíveis
Evidência na coleção	T04 - Cardápio da unidade

ID	T05
Cenário / endpoint	Rota protegida sem token — GET /pedidos
Pré-condição e entrada	Nenhum token
Resultado esperado	401 e envelope UNAUTHENTICATED
Evidência na coleção	T05 - Rota protegida sem token

ID	T06
Cenário / endpoint	Login administrativo — POST /auth/login
Pré-condição e entrada	Credenciais de seed
Resultado esperado	200 e token ADMIN
Evidência na coleção	T06 - Login do administrador do seed

ID	T07
Cenário / endpoint	Criar COZINHA — POST /auth/usuarios-operacionais
Pré-condição e entrada	Token ADMIN
Resultado esperado	201 e perfil COZINHA
Evidência na coleção	T07 - Criar usuário de cozinha

ID	T08
Cenário / endpoint	Criar ATENDENTE — mesma rota
Pré-condição e entrada	Token ADMIN
Resultado esperado	201 e perfil ATENDENTE
Evidência na coleção	T08 - Criar usuário de atendimento

ID	T09
Cenário / endpoint	Criar produto — POST /produtos
Pré-condição e entrada	Token ADMIN
Resultado esperado	201 e preço R\$ 100,00
Evidência na coleção	T09 - Criar produto de teste para fluxo e pontos

ID	T10
Cenário / endpoint	Entrada de estoque — POST /unidades/{id}/estoque/{id}/movimentacoes
Pré-condição e entrada	Unidade do seed, produto T09, token ADMIN
Resultado esperado	200 e saldo 20
Evidência na coleção	T10 - Entrada inicial de estoque

ID	T12
Cenário / endpoint	Campo obrigatório ausente — POST /pedidos
Pré-condição e entrada	Cliente autenticado; request sem canalPedido
Resultado esperado	422 VALIDATION_ERROR
Evidência na coleção	T12 - Canal obrigatório ausente

ID	T13
Cenário / endpoint	Produto inexistente — POST /pedidos
Pré-condição e entrada	Cliente autenticado; produtoId=9999999999
Resultado esperado	404 e erro padronizado
Evidência na coleção	T13 - Produto inexistente

ID	T14
Cenário / endpoint	Estoque insuficiente — POST /pedidos
Pré-condição e entrada	Cliente autenticado; quantidade 1000
Resultado esperado	409 e estoque inalterado
Evidência na coleção	T14 - Estoque insuficiente

ID	T14B
Cenário / endpoint	Quantidade negativa — POST /pedidos
Pré-condição e entrada	Cliente autenticado; quantidade -1
Resultado esperado	422 VALIDATION_ERROR
Evidência na coleção	T14B - Quantidade negativa

ID	T15
Cenário / endpoint	Criar pedido — POST /pedidos
Pré-condição e entrada	Cliente, produto e unidade válidos; canal TOTEM
Resultado esperado	201, status AGUARDANDO_PAGAMENTO e canalPedido=TOTEM
Evidência na coleção	T15 - Criar pedido válido

ID	T15B
Cenário / endpoint	Filtrar canal — GET /pedidos?canalPedido=TOTEM
Pré-condição e entrada	Pedido T15
Resultado esperado	200 e pedido presente
Evidência na coleção	T15B - Filtrar pedidos por canalPedido

ID	T16
Cenário / endpoint	Pagamento aprovado — POST /pedidos/{id}/pagamento-mock
Pré-condição e entrada	Pedido T15, resultado APROVADO
Resultado esperado	200, pagamento MOCK e pedido EM_PREPARACAO
Evidência na coleção	T16 - Aprovar pagamento mock

ID	T17
Cenário / endpoint	Perfil sem permissão — PATCH /pedidos/{id}/status
Pré-condição e entrada	Cliente tenta mudar pedido em preparação
Resultado esperado	403
Evidência na coleção	T17 - Cliente não pode avançar status

ID	T18
Cenário / endpoint	Pagamento repetido — mesma rota de pagamento
Pré-condição e entrada	Pedido já aprovado
Resultado esperado	409; não cria outro pagamento
Evidência na coleção	T18 - Pagamento repetido é bloqueado

ID	T19
Cenário / endpoint	Login COZINHA — POST /auth/login
Pré-condição e entrada	Conta T07
Resultado esperado	200 e token COZINHA
Evidência na coleção	T19 - Login da cozinha

ID	T20
Cenário / endpoint	Pedido pronto — PATCH /pedidos/{id}/status
Pré-condição e entrada	Pedido em preparação, token COZINHA
Resultado esperado	200 e status PRONTO
Evidência na coleção	T20 - Cozinha marca pedido pronto

ID	T21
Cenário / endpoint	Login ATENDENTE — POST /auth/login
Pré-condição e entrada	Conta T08
Resultado esperado	200 e token ATENDENTE
Evidência na coleção	T21 - Login do atendente

ID	T22
Cenário / endpoint	Entregar pedido — PATCH /pedidos/{id}/status
Pré-condição e entrada	Pedido PRONTO, token ATENDENTE
Resultado esperado	200 e status ENTREGUE
Evidência na coleção	T22 - Atendente entrega pedido

ID	T24
Cenário / endpoint	Segundo pedido — POST /pedidos
Pré-condição e entrada	Cliente com pelo menos 100 pontos
Resultado esperado	201 e status AGUARDANDO_PAGAMENTO
Evidência na coleção	T24 - Criar pedido para resgatar pontos

ID	T25
Cenário / endpoint	Resgatar pontos — POST /fidelidade/resgates
Pré-condição e entrada	Pedido T24; 100 pontos disponíveis
Resultado esperado	200, desconto R\$ 5,00 e total R\$ 95,00
Evidência na coleção	T25 - Resgatar 100 pontos

ID	T26
Cenário / endpoint	Pagamento recusado — rota de pagamento mock
Pré-condição e entrada	Pedido T24 com resgate
Resultado esperado	200, status CANCELADO, pagamento RECUSADO; estoque e pontos estornados
Evidência na coleção	T26 - Recusar pagamento e estornar pontos

ID	T27
Cenário / endpoint	Consultar saldo — GET /fidelidade/me
Pré-condição e entrada	Cliente autenticado
Resultado esperado	200 e saldo restaurado a 100 pontos
Evidência na coleção	T27 - Conferir estorno no saldo de fidelidade

ID	T29
Cenário / endpoint	Evidência de auditoria — GET /auditoria
Pré-condição e entrada	Token ADMIN; filtro pelo pedido T15
Resultado esperado	200 e evento PEDIDO_CRIADO
Evidência na coleção	T29 - Evidenciar criação de pedido na auditoria

O conjunto contém mais que os 10 cenários mínimos e cobre casos positivos e negativos, autenticação, autorização, validação, estoque, pagamento recusado, canal do pedido e auditoria. A execução local da coleção foi confirmada em 25/09/2026: 35 chamadas, 35 scripts de teste e nenhuma falha. O resumo está em [evidência de testes](#).

Regressão de validações e documentação

ID	T30
Cenário / endpoint	Nome curto — POST /produtos
Pré-condição e entrada	ADMIN autenticado; nome " a ", preço 10
Resultado esperado	422; mensagem em português; mínimo dois caracteres após normalização
Evidência na coleção	T30 - Rejeitar nome curto após remover espaços

ID	T31
Cenário / endpoint	Endereço curto — POST /unidades
Pré-condição e entrada	ADMIN autenticado; nome válido e endereço " a "
Resultado esperado	422; mínimo cinco caracteres após normalização
Evidência na coleção	T31 - Rejeitar endereço curto após remover espaços

ID	T32
Cenário / endpoint	E-mail inválido — POST /auth/cadastro
Pré-condição e entrada	Nome e senha válidos; e-mail "invalido"
Resultado esperado	422; mensagem de e-mail em português
Evidência na coleção	T32 - Validar e-mail com mensagem em português

ID	T33
Cenário / endpoint	Contrato de erros — GET /openapi.json
Pré-condição e entrada	API iniciada
Resultado esperado	200; erros 401, 403, 404, 409 e 422 do pedido usam RespostaErro
Evidência na coleção	T33 - Conferir erros no contrato OpenAPI

Apêndice D — Evidências de execução

Revisão: 25/09/2026.

Ambiente e método

Banco SQLite novo e isolado, preparado com a migração Alembic e o seed. A coleção Postman foi executada por Newman contra a API local. Os dados são fictícios e as credenciais de execução não fazem parte da entrega.

Resultado da coleção

35 requisições executadas.

35 verificações aprovadas.

0 falhas.

Fluxo principal, cenários negativos, fidelidade e auditoria executados.

Quatro cenários de regressão incluídos para normalização de texto, mensagens em português e contrato OpenAPI.

Verificação complementar

Uma rodada independente confirmou 11 casos: nome de produto curto após retirar espaços, endereço curto após retirar espaços, e-mail inválido, senha incorreta, token inválido, paginação inválida, cadastro válido, cadastro duplicado, canal inválido, produto duplicado no pedido e tentativa de cadastro de produto por cliente. Os 11 casos retornaram os códigos esperados. Esses casos complementares não são somados às 35 verificações da coleção.

Reproduzir

Siga o README, aplique a migração e o seed, inicie a API e importe a coleção e o ambiente em `postman/`. Preencha as credenciais locais do administrador e execute as pastas na ordem. Os cenários T30 a T33 dependem do token administrativo obtido no fim do fluxo anterior.

O resultado demonstra somente os comportamentos cobertos. Não representa teste de carga nem certificação de segurança em produção.

Resultado por requisição

Cenário	HTTP observado	Verificação
T01 - Cadastrar cliente com consentimento opcional	201	Passou
T02 - Login do cliente	200	Passou
T03 - Listar unidades e guardar ID do seed	200	Passou
T04 - Cardápio da unidade	200	Passou
T05 - Rota protegida sem token	401	Passou
T06 - Login do administrador do seed	200	Passou
T07 - Criar usuário de cozinha	201	Passou
T08 - Criar usuário de atendimento	201	Passou
T09 - Criar produto de teste para fluxo e pontos	201	Passou
T10 - Entrada inicial de estoque	200	Passou
T11 - Login do cliente antes do fluxo	200	Passou
T12 - Canal obrigatório ausente	422	Passou
T13 - Produto inexistente	404	Passou
T14 - Estoque insuficiente	409	Passou
T14B - Quantidade negativa	422	Passou
T15 - Criar pedido válido	201	Passou
T15B - Filtrar pedidos por canalPedido	200	Passou
T16 - Aprovar pagamento mock	200	Passou
T17 - Cliente não pode avançar status	403	Passou
T18 - Pagamento repetido é bloqueado	409	Passou
T19 - Login da cozinha	200	Passou
T20 - Cozinha marca pedido pronto	200	Passou
T21 - Login do atendente	200	Passou
T22 - Atendente entrega pedido	200	Passou
T23 - Login do cliente para segundo pedido	200	Passou
T24 - Criar pedido para resgatar pontos	201	Passou
T25 - Resgatar 100 pontos	200	Passou

Cenário	HTTP observado	Verificação
T26 - Recusar pagamento e estornar pontos	200	Passou
T27 - Conferir estorno no saldo de fidelidade	200	Passou
T28 - Login administrativo para consultar auditoria	200	Passou
T29 - Evidenciar criação de pedido na auditoria	200	Passou
T30 - Rejeitar nome curto após remover espaços	422	Passou
T31 - Rejeitar endereço curto após remover espaços	422	Passou
T32 - Validar e-mail com mensagem em português	422	Passou
T33 - Conferir erros no contrato OpenAPI	200	Passou

Apêndice E — Decisões técnicas

Organização da aplicação

O código está separado por responsabilidade: `api` recebe e valida as requisições HTTP; `application` coordena os casos de uso; `domain` guarda os estados e tipos do negócio; `infrastructure` configura persistência e modelos do banco. Os contratos de entrada e saída ficam em `application/contracts.py`, para que os casos de uso não precisem importar os módulos das rotas.

As setas representam chamadas e uso de dados. A API traduz HTTP para chamadas da aplicação e converte erros de negócio em respostas HTTP padronizadas. As rotas não devem conter regras de estoque, pagamento ou fidelidade.

Escopo do MVP

O fluxo demonstrável é pedido → pagamento simulado → preparação → entrega. A API persiste usuários, unidades, produtos, saldos por unidade, pedidos, itens, pagamentos e auditoria em SQLite. `canalPedido` é obrigatório, validado como enum e disponível como filtro.

O preço dos itens é copiado para o pedido no momento da compra. O pedido desconta o saldo durante a espera pelo pagamento; uma recusa restaura o estoque. Transições aceitas: AGUARDANDO_PAGAMENTO → EM_PREPARACAO → PRONTO → ENTREGUE; pagamento recusado leva a CANCELADO.

Segurança e privacidade

Dados pessoais do cliente: nome e e-mail; finalidade operacional: autenticação, identificação da conta e atendimento de pedidos. A hipótese legal para operações necessárias ao pedido deve ser validada pelo controlador; execução de contrato é uma hipótese prevista no art. 7º, V, da LGPD.

Fidelidade é opcional. Seu consentimento é registrado com data e pode ser revogado sem impedir o cadastro ou a realização de pedidos. A adesão usa a hipótese de consentimento do art. 7º, I, e deve permanecer ligada à finalidade específica do programa.

Senhas são armazenadas com hash Argon2. Tokens têm prazo de validade e endpoints verificam o perfil. A API mock não coleta nem armazena dados de cartão.

A auditoria guarda identificador do ator, ação, tipo/ID do registro e horário; não deve guardar senha, token, dados de cartão ou conteúdo pessoal desnecessário.

O controlador deve definir prazo de retenção por categoria. Ao encerrar a finalidade e não haver obrigação legal ou necessidade de exercício regular de direitos, avaliar eliminação ou anonimização. A implementação acadêmica não substitui uma política aprovada pelo controlador.

Regras de fidelidade implementadas

O cliente escolhe participar; o sistema guarda cada aceite ou revogação para preservar o histórico.

Com consentimento ativo, cada R\$ 1,00 efetivamente pago gera 1 ponto.

A cada 100 pontos resgatados, o pedido recebe R\$ 5,00 de desconto, desde que reste valor positivo para pagar.

Resgate só ocorre em pedido próprio aguardando pagamento. Se o pagamento for recusado, os pontos resgatados são estornados.

Promoções (proposta, ainda não implementada)

Uma campanha deve ter período de validade, unidades e produtos elegíveis, regra de desconto e estado ativo. O back-end deve avaliar a campanha, calcular o valor e registrar no pedido o desconto aplicado e a campanha correspondente. O cliente nunca informa o valor final do desconto.

Persistência e migrações

Alembic controla versões do esquema. Para criar uma mudança, gerar uma revisão com `alembic revision --autogenerate -m "descricao"`, revisar o arquivo gerado e aplicar com `alembic upgrade head`. A aplicação não cria tabelas automaticamente ao iniciar.

Referências de privacidade

BRASIL. Lei nº 13.709, de 14 de agosto de 2018 (LGPD), especialmente arts. 6º, 7º, 8º e 18. [Texto compilado no Planalto](#).

AUTORIDADE NACIONAL DE PROTEÇÃO DE DADOS. [Direitos dos titulares](#).